

SkillClaw: Let Skills Evolve Collectively with Agentic Evolver



Ziyu Ma*, Shidong Yang*, Yuxiang Ji*, Xucong Wang*, Yong Wang†, Yiming Hu, Tongwen Huang, Xiangxiang Chu

DreamX Team, Alibaba Group *Equal contribution †Project lead

Large language model (LLM) agents such as OpenClaw rely on reusable skills to perform complex tasks, yet these skills remain largely static after deployment. As a result, similar workflows, tool usage patterns, and failure modes are repeatedly rediscovered across users, preventing the system from improving with experience. While interactions from different users provide complementary signals about when a skill works or fails, existing systems lack a mechanism to convert such heterogeneous experiences into reliable skill updates. To address these issues, we present SkillClaw, a framework for collective skill evolution in multi-user agent ecosystems, which treats cross-user and over-time interactions as the primary signal for improving skills. SkillClaw continuously aggregates trajectories generated during use and processes them with an autonomous evolver, which identifies recurring behavioral patterns and translates them into updates to the skill set by refining existing skills or extending them with new capabilities. The resulting skills are maintained in a shared repository and synchronized across users, allowing improvements discovered in one context to propagate system-wide while requiring no additional effort from users. By integrating multi-user experience into ongoing skill updates, SkillClaw enables cross-user knowledge transfer and cumulative capability improvement, and experiments on WildClawBench show that limited interaction and feedback, it significantly improves the performance of Qwen3-Max in real-world agent scenarios.

Github: <https://github.com/AMAP-ML/SkillClaw>

1 Introduction

Large language model (LLM) agents (Yao et al., 2022; Shinn et al., 2023) have rapidly made personal AI assistants practical in real-world settings, with systems such as OpenClaw enabling users to complete complex tasks through natural conversation. A user can now ask an agent to configure a service, debug an API call, or automate a multi-step workflow, relying on it to coordinate tool usage and intermediate reasoning. These capabilities are largely driven by skills, which encode structured procedures for interacting with tools and solving tasks. In current deployments, users typically select and install skills from a centralized skill hub to meet their needs, and these skills serve as the primary building blocks for agent behavior. However, the skill ecosystem remains largely static (Zhang et al., 2025b; Naihin et al., 2023; Song et al., 2026), as skills are manually installed and maintained and solutions discovered during interaction rarely persist beyond individual sessions.

This limitation becomes evident in everyday usage. For example, users often ask agents to complete multi-step tasks such as automating data processing workflows, where failures frequently arise from subtle issues such as incorrect argument formats or mismatched tool calls. Through several rounds of trial and error, an agent may eventually arrive at a working solution or even a more stable procedure. However, these improvements remain confined to the current session and are not consolidated into the skill set or carried forward to future interactions. As similar tasks recur across different users and over time, the same patterns of failure and recovery are repeatedly observed, yet the system does not improve its behavior. This is fundamentally problematic because users operate in overlapping task spaces where similar workflows, tools, and failure modes are shared, but the system fails to leverage these recurring experiences. Consequently, each user is forced to rediscover solutions independently, preventing knowledge from accumulating at the system level. Therefore, the key challenge is not only to improve performance within a single session, but also to enable

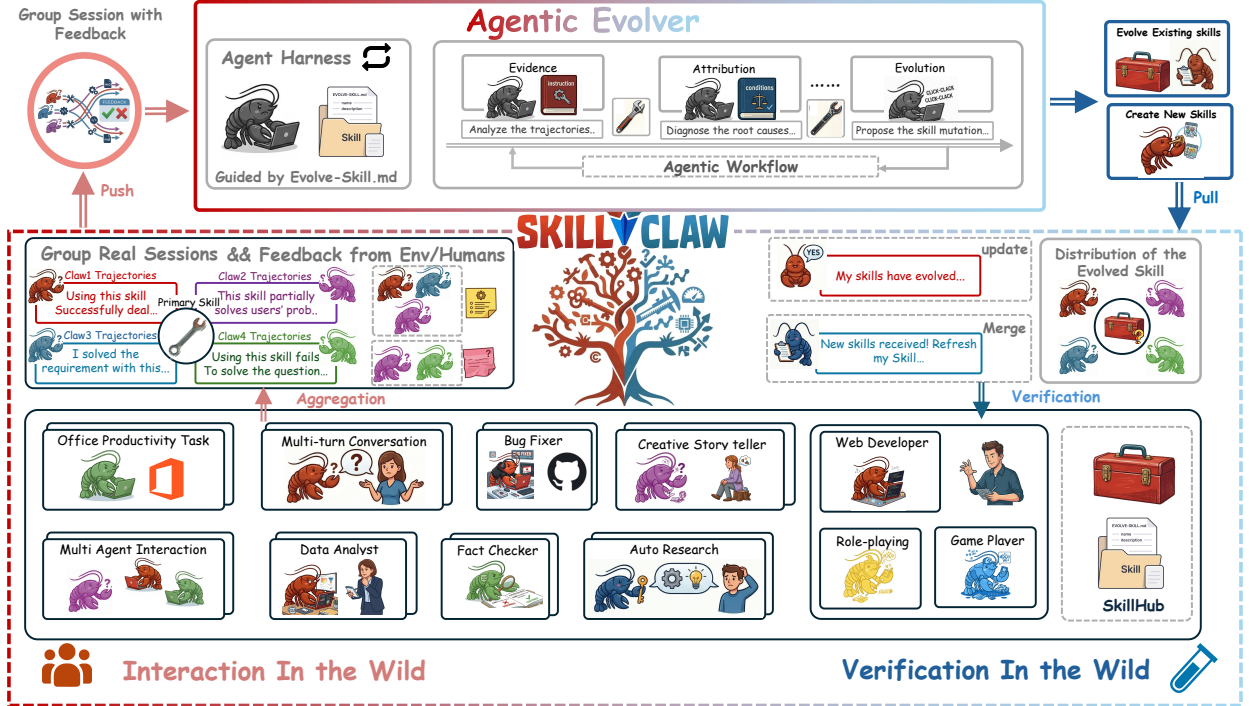


Figure 1 Overview of SkillClaw. SkillClaw enables collective skill evolution in a multi-user agent ecosystem through a closed-loop pipeline. Independent agents interact with their environments and produce structured session trajectories that preserve full action–feedback causal chains. These trajectories are aggregated across users and grouped by referenced skills, forming a shared evidence base that exposes consistent success patterns and recurring failure modes. An agentic evolver analyzes each skill-specific group and performs evidence-driven updates via refinement or creation, while preserving validated behaviors from successful executions. The updated skill repository is then synchronized back to all agents, allowing improvements discovered in one user’s interaction to benefit others and continuously accumulate over time.

knowledge to accumulate and evolve across users.

Existing approaches to agent adaptation fail to support the accumulation and evolution of skills across users and over time. Memory-based methods store past trajectories for retrieval (Shinn et al., 2023; Zhao et al., 2024; Fang et al., 2025a; Tang et al., 2025; Ouyang et al., 2025a; Chhikara et al., 2025; Liu et al., 2026), but such records remain tied to specific instances and are difficult to generalize into improved behavior. Skill-based methods compress experience into structured instructions (Xia et al., 2026a; Zhang et al., 2025a, 2026b; Wu et al., 2025; Zhang et al., 2026a), yet treat the resulting skill library as a static resource that does not evolve through usage. While local refinement can improve individual agent instances, these improvements remain isolated and do not accumulate across users, leading to fragmented skills rather than collective improvement over time. What is missing is a mechanism that turns ordinary interactions into continuous skill evolution and enables skills to improve collectively across users.

Building on this insight, we propose **SkillClaw**, a framework for skill collective evolution in multi-user OpenClaw-style agent ecosystems (Fig 1). SkillClaw adopts a centralized evolution architecture, where agents deployed across different users continuously generate interaction sessions during everyday usage. These trajectories are aggregated across users and over time as evidence of real-world task execution and are processed by a centralized evolution engine to drive skill updates. Given accumulated interaction trajectories, the evolver analyzes both successful and failed executions, identifies recurring issues and effective procedures, and updates the shared skill set by refining existing skills, creating new ones, or adjusting their descriptions. Unlike predefined pipelines, this evolution process is driven by an autonomous agent that performs open-ended reasoning over interaction evidence and directly edits skill definitions. The updated skills are then synchronized across agents, allowing improvements discovered in one context to propagate to future interactions across users and over time. This forms a continuous evolution loop in which interaction data drives skill updates, and updated skills improve subsequent interactions. From the user’s perspective, this process

requires no additional effort, as data collection, evolution, and synchronization all occur automatically in the background.

This design introduces three key properties that distinguish SkillClaw from existing systems. First, SkillClaw enables *collective evolution*, where knowledge from individual interactions contributes to a shared and continuously improving skill ecosystem. Second, it is *fully automatic*, with skill evolution driven by runtime interaction without manual curation or explicit user intervention. Third, it adopts an *agentic evolution paradigm*, where skill updates are produced through open-ended reasoning rather than predefined update rules, enabling flexible and context-aware improvements.

SkillClaw is designed as a general framework that is compatible with a wide range of Claw-style agent systems, including OpenClaw as well as variants such as CoPaw, IronClaw, PicoClaw, ZeroClaw, NanoClaw, and NemoClaw. We evaluate SkillClaw on WildClawBench using `qwen3-max` as the backbone model and simulate a multi-user deployment setting. Experimental results demonstrate that SkillClaw yields substantial improvements across tasks, highlighting the effectiveness of multi-user driven collective evolution for building continuously improving agent systems in real-world environments.

2 Method

We present **SkillClaw**, a framework for collective skill evolution in a multi-user agent ecosystem (Fig 1). In our setting, different users independently interact with their own deployed OpenClaw agents, potentially across different devices, environments, and time. Although these interactions are isolated at runtime, they share a common behavioral space: similar workflows, overlapping tool usage, and recurring failure modes appear across users. SkillClaw builds on the observation that *different users exercising the same skill under diverse contexts produce complementary views of that skill’s behavioral boundary*, revealing both the conditions under which it works and those under which it breaks. A single user rarely generates enough signal to separate a generalizable improvement from an idiosyncratic fix. Aggregating evidence across users provides the grounding that makes stable skill evolution possible.

Formally, let $\mathcal{S} = \{s_1, \dots, s_M\}$ denote a shared skill set, where each skill is a reusable procedural artifact. Each user interaction produces a session trajectory τ , which records the full interaction loop: the prompt, the agent’s actions, feedback from the environment or the user, and the final agent response. Given a set of trajectories $\mathcal{T} = \{\tau_i\}$ collected across users, our goal is to update the shared skill set:

$$\mathcal{S}' = \Phi(\mathcal{S}, \mathcal{T}),$$

such that improvements discovered in one interaction can benefit future users.

2.1 From Isolated Sessions to Shared Evidence

Multi-user skill evolution requires converting a stream of isolated, heterogeneous interaction sessions into a form that supports cross-user reasoning. SkillClaw does this in two stages: it first structures individual sessions to preserve causal information, then aggregates them into a shared evidence base.

At the system level, SkillClaw connects independently deployed agents through a common skill repository. Each agent has access to the current skill set and produces interaction sessions during normal usage. These sessions are recorded and uploaded as shared evidence. A centralized evolution engine periodically processes the collected sessions, updates the skill repository, and synchronizes the updated skills back to all agents, forming a closed loop:

$$\text{Multi-user Interaction} \rightarrow \text{Session Collection} \rightarrow \text{Skill Evolution} \rightarrow \text{Skill Synchronization}.$$

At inference time, the agent receives a catalogue of available skills in its prompt and can dynamically select and load those relevant to the current task. Users do not interact directly, and no coordination among agents is required. Collective improvement arises entirely from shared skill evolution.

Within this loop, each interaction session contains more than plain dialogue. SkillClaw records the full causal chain: the user prompt, the agent’s actions (including tool calls), intermediate feedback (tool results, errors,

Algorithm 1 Agentic Collective Skill Evolution

Require: Skill repository $\mathcal{S}$, user sessions $\mathcal{T}$

Ensure: Updated repository $\mathcal{S}'$

- 1: Convert $\mathcal{T}$ into structured evidence $\mathcal{E}$
 - 2: Group $\mathcal{E}$ by referenced skills to obtain $\{\mathcal{G}(s)\}$ and $\mathcal{G}(\emptyset)$
 - 3: $\mathcal{S}' \leftarrow \mathcal{S}$
 - 4: **for all** group $\mathcal{G}(s)$ **do**
 - 5: Use the agentic evolver to analyze recurring success and failure patterns
 - 6: Select an evolution action from $\{\text{refine}, \text{create}, \text{skip}\}$
 - 7: Generate a candidate skill update if the evidence supports modification
 - 8: Apply conservative editing and validation
 - 9: Merge approved updates into $\mathcal{S}'$
 - 10: **end for**
 - 11: Analyze $\mathcal{G}(\emptyset)$ for missing but reusable procedures
 - 12: Add validated new skills into $\mathcal{S}'$
 - 13: Synchronize $\mathcal{S}'$ back to all agents
 - 14: **return** $\mathcal{S}'$
-

and explicit user responses), and the final agent response. We record all of this because most skill-level failures are *procedural*. An incorrect argument format, a missing validation step, or a misordered tool call can cause a task to fail, yet none of these problems appears in the final response. They can only be diagnosed from the intermediate action-feedback trace. Each raw session is converted into a structured representation that preserves this chain:

$$\text{prompt} \rightarrow \text{action} \rightarrow \text{feedback} \rightarrow \dots \rightarrow \text{agent response}.$$

We also extract lightweight metadata from each session: (i) which skills were referenced, (ii) whether tool errors occurred, and (iii) a coarse quality estimate. These signals help organize sessions but do not impose rigid labels.

Once sessions are structured, they are grouped by the skills they reference to enable cross-user reasoning. For each skill s , we collect all sessions that invoked s :

$$\mathcal{G}(s) = \{\tau_i \mid s \in \mathcal{K}_i\},$$

and place sessions that did not use any skill into a separate group $\mathcal{G}(\emptyset)$. This grouping does more than organize the data. When multiple sessions invoke the same skill but produce different outcomes across different users, tasks, or environments, the comparison directly reveals where the skill works and where it fails, with the skill itself as the controlled factor. This amounts to a *natural ablation* and enables two operations that would be unreliable from single-user data alone: (1) evaluating how an existing skill actually performs under diverse real-world usage, and (2) identifying recurring procedures that no existing skill covers, surfaced by patterns in $\mathcal{G}(\emptyset)$.

2.2 Agentic Skill Evolution

The core of SkillClaw is an *agentic evolver* that updates the shared skill repository with open-ended reasoning. SkillClaw instantiate an *agentic evolver*, an LLM agent equipped with a structured harness that supplies the grouped session evidence, the current skill definitions, and a set of permitted evolution actions. The harness provides structured inputs but does not constrain the evolver’s reasoning. The evolver diagnoses root causes from sessions of varying context lengths and skills of different formats, and decides how to act. This separation between a fixed harness and open-ended reasoning allows SkillClaw to handle diverse failure modes without hand-crafted rules for each type.

Concretely, given a skill s and its associated session group $\mathcal{G}(s)$, the evolver examines both successful and failed executions and selects one of three actions:

- **Refine.** Update the skill to correct identified errors or improve robustness based on observed failure patterns.
- **Create.** Introduce a new skill when $\mathcal{G}(s)$ reveals recurring sub-procedures that are not captured by any existing skill.
- **Skip.** Leave the skill unchanged when the available evidence is insufficient to justify a modification.

For sessions in $\mathcal{G}(\emptyset)$, i.e., those that did not invoke any skill, the evolver focuses on discovering missing but reusable procedures. New skills are created only when the observed patterns are specific enough to be teachable and likely to recur.

Regardless of which action is chosen, the evolver always reasons over successful and failed sessions *jointly*. Successful sessions define the *invariants* of a skill, the parts that work and must not be altered. Failed sessions define the *targets*, the specific behaviors that need correction. This joint view is what prevents a naive failure: fixing one problem while inadvertently breaking a previously effective procedure. Each update corrects identified deficiencies while preserving what successful sessions have validated, making evolution cumulative. The complete procedure is given in Algorithm 1.

2.3 Skill Synchronization and the Evolution Loop

After evolution, candidate skill updates are validated before being written back to the shared repository. Validation is performed during the nighttime and executed in available idle user environments, ensuring that evaluation reflects real deployment conditions. For a skill s and its candidate update s' , the system selects relevant tasks from the interaction data collected during the day. Both versions are executed under the same environment using the full toolchain, including multi-step interactions and intermediate feedback. After execution, the system uses the model to compare the outcomes produced by s and s' . The decision is based on overall task success and execution stability. If the updated skill demonstrates better performance, it is marked as **Accept**; otherwise, it is marked as **Reject**. Accepted updates are merged into the shared repository and synchronized to all agents for the next day. Rejected updates are retained only as candidates and are not deployed. As a result, users always interact with the best validated skill pool from the previous night, rather than unverified updates. This validation step induces a monotonic deployment behavior. Since only improvements are accepted, the deployed skill pool does not degrade over time. Combined with the evolution process, the system forms a closed loop:

$$\text{Interaction} \rightarrow \text{Evidence} \rightarrow \text{Evolution} \rightarrow \text{Validation} \rightarrow \text{Deployment}.$$

where updated skills shape future interactions and generate new evidence for the next round of evolution.

Three properties follow from this design. First, *collective evolution*. Sessions are aggregated across users, and knowledge discovered in one interaction is propagated to a shared skill ecosystem that benefits all users. Second, *full automation*. The entire pipeline, from session recording to skill synchronization, runs without manual curation or explicit user intervention. The only human input is normal agent usage. Third, *agentic adaptability*. Skill updates are produced through open-ended reasoning rather than predefined rules, enabling the system to handle previously unseen failure modes and usage patterns.

From the user’s perspective, none of this is visible. Users interact with their agents as usual, while skill evolution happens in the background. Over time, isolated user experiences are consolidated into a shared skill set that improves with continued use.

3 Experiments

3.1 Benchmark: WildClawBench

We evaluate SkillClaw on **WildClawBench** (Ding et al. (2026)), a real-world agent benchmark consisting of 60 complex tasks across six capability domains. As summarized in Table 1, the benchmark covers diverse scenarios including productivity workflows, code execution, social interaction, retrieval, creative generation,

Table 1 Task categories in WildClawBench. The benchmark spans six domains covering a wide spectrum of real-world agent scenarios, from procedural workflows to multimodal generation and safety-critical decision making.

Category	Example Tasks	Challenges
Productivity Flow	arXiv classification, scheduling, SCP	multi-step pipelines
Code Intelligence	debugging, puzzle solving	execution correctness
Social Interaction	negotiation, chat analysis	multi-turn reasoning
Search & Retrieval	academic search, conflict resolution	API usage
Creative Synthesis	video notes, poster generation	multimodal generation
Safety & Alignment	prompt injection, leakage detection	constraint satisfaction

Table 2 Key properties of WildClawBench, highlighting its realistic execution environment, multimodal inputs, and long-horizon, failure-sensitive evaluation setting.

Property	Description
Execution Environment	Full Linux container with tools
Multimodality	Text, code, image, video
Evaluation	3–27 metrics aggregated
Hard Constraints	Critical errors $\rightarrow$ zero score
Task Length	15–50 steps
External Dependency	APIs and model downloads

and safety alignment. Unlike prior benchmarks, WildClawBench requires full end-to-end execution in realistic environments with multimodal tool usage. Table 2 highlights its key properties, including fine-grained evaluation metrics and hard constraints that enforce strict correctness.

3.2 Experimental Setup

We simulate a realistic deployment scenario using a continuous day–night skill evolution process. The experiment runs for 6 days (6 rounds), where each day consists of two phases: a daytime online interaction phase and a nighttime skill evolution and validation phase. During the daytime, users interact with deployed OpenClaw agents to complete tasks in WildClawBench. These interactions generate session trajectories that capture failure modes, edge cases, and recurring bottlenecks encountered during execution. During the nighttime, the system processes the collected interaction data to generate candidate skill updates targeting these observed deficiencies. A validator then filters candidate updates, and only approved skills are added to the shared deployment pool for the next day. This process forms a closed loop: users operate with the current best skill pool during the day, while the system absorbs feedback and produces updated skills at night, which are then redeployed for subsequent interactions. Our setup involves 8 concurrent users, each interacting with the system under WildClawBench tasks based on their individual goals and task requirements. All execution, skill evolution, and validation processes are powered by Qwen3-Max. At the system level, we maintain a shared current best skill pool. Day 1 starts with an initial skill set corresponding to the baseline. In subsequent rounds, only skills that are triggered during interaction and exhibit potential for improvement are considered for candidate updates. Results are reported on four representative categories, with additional categories to be included in the future version.

Validation Mechanism. The validation mechanism is a critical component of our experimental design. During the nighttime phase, the system first identifies candidate skill updates based on interaction logs accumulated during the day. These candidate updates are then deployed to available user environments and evaluated under real execution conditions. The validator follows a simple decision rule. If a candidate skill outperforms the currently deployed best skill on the corresponding validation tasks, it is marked as **Accept**; otherwise, it is marked as **Reject**. Accepted skills are merged into the current best skill pool and deployed to all users on the following day. Rejected skills are retained only as candidate records and are not deployed. As a result, users always interact with the best validated skill pool from the previous night, rather than unverified updates. This validation strategy introduces additional token cost, as candidate skills must be

executed in real environments with full tool interaction. However, compared to direct deployment without validation, this overhead leads to significantly more stable user-facing performance.

Table 3 User-side daytime results (best-skill deployment view). Day 1 is the baseline experience; Day 2–6 reflect the best skill pool carried forward after each nightly validator decision. Absolute and relative gains are computed w.r.t. Day 1.

Category	Day 1	Day 2	Day 3	Day 4	Day 5	Day 6	Abs. Gain	Rel. Gain
Social Interaction	54.01%	60.34%	60.34%	60.34%	60.34%	60.34%	+6.33	+11.72%
Search & Retrieval	22.73%	30.00%	30.00%	34.55%	34.55%	34.55%	+11.82	+52.00%
Creative Synthesis	11.57%	21.80%	21.80%	21.80%	21.80%	21.80%	+10.23	+88.41%
Safety & Alignment	24.00%	24.00%	24.00%	24.00%	32.00%	32.00%	+8.00	+33.33%

3.3 Main Results

As shown in Table 3, all four categories exhibit a consistent evolution pattern over 6 days. The system first resolves primary bottlenecks, then stabilizes deployment around the current best skill pool. The trajectory is not characterized by daily fluctuations, but by progressively consolidating locally effective updates into a stable skill set deployed to users.

Social Interaction improves earliest and most sharply. Performance increases from 54.01% to 60.34% on Day 2 and remains stable thereafter. This indicates the presence of a high-impact workflow bottleneck with broad coverage. Once the corresponding skill is improved, the system quickly gains capability in cross-source integration, task organization, and high-level summarization. Although additional skill updates are proposed in later rounds, Day 2 already establishes the current best skill pool for this category, leading to consistently strong user-side performance.

Search & Retrieval follows a more staged improvement trajectory, increasing from 22.73% to 30.00%, and then further to 34.55%. Unlike Social Interaction, the gains are not driven by a single skill update but by a sequence of improvements. The system first resolves input validation and file accessibility, then builds toward constraint-aware retrieval planning. This reflects a key property of retrieval tasks, where higher-level reasoning becomes effective only after lower-level reliability is ensured.

Creative Synthesis shows a large early jump from 11.57% to 21.80% on Day 2 and then plateaus. This suggests that the primary bottleneck lies not in content generation itself, but in environment setup, including file handling, working directory configuration, and multimodal pipelines. Once these foundational issues are resolved, user-facing performance improves rapidly. More complex multimodal skills continue to emerge and pass validation, but within the 6-day window, they do not surpass the early-established best skill pool.

Safety & Alignment improves later, from 24.00% to 32.00%. Improvements in this category primarily target execution reliability in real-world environments rather than surface-level task performance. Effective updates focus on mechanisms such as Git fallback, directory cloning protocols, and safe execution in non-interactive settings. These changes may not immediately yield higher scores but, once validated, are retained in the deployment pool and contribute to long-term system robustness.

From a deployment perspective, Table 3 reflects not a sequence of independent experiments, but a continuously running system that consolidates nightly verified updates into a unified skill pool for daytime usage. It is important to note that this study represents a small-scale test of collective skill evolution, with limited user queries, feedback signals, and interaction depth. Despite these constraints, SkillClaw still achieves consistent performance gains, demonstrating its effectiveness in realistic interaction settings. Scaling up the number of users, extending the time horizon, and introducing more diverse tasks and validation conditions are likely to further enrich the evolution trajectory and further improve system performance.

Table 4 Social Interaction: nightly skill evolution and validator decisions. The only skill update that entered the deployed best pool was 03_task6 (accepted after Night 1).

Day	Candidate Skill	Skill Function	Change Summary	Validator	Next-Day Action
1	03_task6	Cross-dept Slack summarization, data reconciliation, risk identification, board-level brief drafting	Rewrote workflow into strictly-ordered steps; strengthened project keyword filtering, finance priority, change detection, COO contact confirmation	Accept	Day 2: upgrade to new best pool
2	(none)	Continued using current Social best pool	Same-pool retest; no new skill text landed	Reject	Day 3: keep Day 2 best pool
3	03_task1	Gmail + Calendar meeting coordination	Extended workflow with meeting-param extraction, multi-participant availability check, confirmation loop, reschedule on rejection	Reject	Not admitted; Day 4 keeps current best pool
4	(none)	Continued using current Social best pool	Same-pool retest; no new skill text landed	Reject	Day 5: keep current best pool
5	(none)	Continued using current Social best pool	Same-pool retest; no new skill text landed	Reject	Day 6: keep current best pool
6	03_task3	Slack feasibility analysis	Added fallback & grounding constraints; analysis must rely on real API results or user-provided context	Reject	Not admitted to next cycle

3.4 Analysis

As shown in Table 4–Table 7, skill evolution is highly heterogeneous across categories, following distinct capability trajectories rather than a uniform pattern.

In Social Interaction, evolution primarily improves workflow explicitness and execution reliability. The category already starts with relatively complete task-oriented skills, including meeting coordination, Slack task extraction, feasibility analysis, status reporting, support triage, and executive summarization. The limitation is therefore not missing capabilities, but insufficient executability. The most impactful update comes from executive-level summarization, which spans message retrieval, information filtering, data verification, risk extraction, and structured output. Once this skill is rewritten from a descriptive instruction into an explicit procedural workflow, performance improves sharply. Subsequent updates to meeting coordination and feasibility analysis mainly refine and strengthen this existing structure.

Search & Retrieval exhibits a staged evolution pattern. Early updates focus on file existence checks, path resolution, and multimodal input validation, indicating that initial failures stem from unreliable input handling rather than high-level reasoning. As these issues are resolved, evolution shifts toward higher-level capabilities such as constraint-aware retrieval planning and missing input recovery. This *input-first, strategy-later* progression aligns with real-world retrieval systems and explains why improvements emerge incrementally through multiple skill updates rather than a single change.

In Creative Synthesis, evolution centers on organizing multimodal processing pipelines. Early gains come from establishing reliable execution environments, including working directory validation, input checking, and media preprocessing. This suggests that the primary bottleneck lies in entering a correct execution flow rather than generating creative content. Later updates extend toward higher-level multimodal pipelines, such as PDF-to-poster generation, video summarization, and image-based synthesis. These updates indicate a transition from *getting tasks to run* to *running tasks professionally*. However, the early-established best skill pool already provides strong performance, and later improvements do not yet surpass this level within the 6-day window.

Safety & Alignment follows a reliability-driven evolution path. Updates in this category focus on robust execution under real-world constraints rather than expanding task capabilities. Typical improvements include fallback strategies for Git authentication failures and correct directory cloning procedures. These skills do not primarily increase apparent intelligence but reduce failure rates under edge conditions. Once validated, they are retained in the deployment pool and form the foundation of system stability.

Overall, Table 4–Table 7 show that skill evolution is not a simple accumulation of rules, but a structured

Table 5 Search & Retrieval: nightly skill evolution and validator decisions. Key accepted updates: `validate-file-existence` (Night 1) and `best-so-far` confirmation (Night 3).

Day	Candidate Skill	Skill Function	Change Summary	Validator	Next-Day Action
1	<code>validate-file-existence</code>	Pre-processing file existence check	Before any file parsing / image reading / multimodal call, first confirm the input file actually exists	Accept	Day 2: upgrade to new best pool
2	<code>debug-missing-file-path</code>	Missing-file path debugging	List parent directory, verify naming, correct path instead of halting on “missing”	Reject	Day 3: keep Day 2 best pool
3	(none)	Continued using current Search best pool	Same-pool retest; nightly readout was stronger, confirming current pool as best-so-far	Accept	Day 4: continue same best pool
4	<code>robust-file-validation-before-multimodal</code>	Stronger multimodal pre-validation	Upgraded from “exists?” to “exists + parent-dir search + hard pre-multimodal validation”	Reject	Day 5: keep current best pool
5	<code>constrained-technical-search-planning</code>	Budget-constrained technical / academic search planning	Added feasibility check, sub-question decomposition, official-source priority, evidence-chain output	Reject	Day 6: keep current best pool
6	<code>recover-missing-input-file</code>	Recover / locate real input file from workspace	When benchmark’s expected path fails, proactively search the working directory for the actual input file	Reject	Not admitted to next cycle

process driven by category-specific bottlenecks. Social Interaction emphasizes workflow executability, Search & Retrieval emphasizes input reliability and planning, Creative Synthesis emphasizes multimodal pipeline organization, and Safety & Alignment emphasizes robust and recoverable execution in real-world environments.

Controlled validation of skill evolution. Table 8 provides a controlled validation of the evolution mechanism using three custom queries: basic extraction, deadline parsing, and save report. Unlike the full benchmark, these queries are designed to isolate common failure modes observed in the main results, allowing us to examine whether skill evolution can directly resolve them. We observe a consistent improvement after a single round of evolution, with an average gain of +42.1%. In particular, *save report* improves from 28.3% to 100.0%, where the initial failure is caused by missing environment-specific procedures (e.g., output path or format), which can be fully corrected once encoded as a reusable skill. Similarly, *basic extraction* shows a large gain (+47.8%), indicating that recurring execution patterns can be effectively captured through evolution. In contrast, *deadline parsing* exhibits a smaller improvement (+6.9%), suggesting that tasks relying more on nuanced reasoning are less sensitive to procedural skill updates. Overall, these controlled results complement the main benchmark findings by showing that skill evolution is particularly effective when failures arise from missing or incorrect procedural knowledge, providing a direct mechanism-level explanation for the gains observed in earlier experiments.

3.5 Case Study

Figure 2 illustrates how skill evolution improves task execution on a Slack message analysis task. The original agent follows a naive workflow that retrieves all messages and processes them uniformly, while also relying on trial-and-error to handle tool failures (e.g., incorrect API port configuration). As a result, execution is both inefficient and error-prone. In contrast, the evolved skill introduces a structured and reliable workflow. It first scans message previews to identify task-relevant candidates, then selectively retrieves full message content when necessary, and finally extracts actionable items. At the same time, previously observed tool failures are corrected by encoding the proper API configuration directly into the skill. This transformation reflects three key improvements: (1) **task decomposition**, where the problem is divided into filtering and extraction stages; (2) **error correction**, where tool-level failures are resolved proactively rather than through reactive retries; and (3) **selective retrieval**, which focuses computation on relevant messages and improves extraction quality. Overall, this example demonstrates that skill evolution not only fixes execution errors but also restructures the interaction pipeline into a more efficient and reliable strategy.

Table 6 Creative Synthesis: nightly skill evolution and validator decisions. The only accepted skill was `validate-tmp-workspace-inputs` (Night 1).

Day	Candidate Skill	Skill Function	Change Summary	Validator	Next-Day Action
1	<code>validate-tmp-workspace-inputs</code>	Check <code>/tmp/workspace</code> inputs & environment setup	Before creative tasks, verify <code>/tmp/workspace</code> inputs, directories, and symlinks are correct	Accept	Day 2: upgrade to new best pool
2	<code>multimodal-input-validation-and-setup</code>	Multimodal input validation & output env init	Check video / image / PDF / audio files exist, are readable, and format-correct; prepare output directories	Reject	Day 3: keep current best pool
3	<code>multimodal-creative-task-pipeline</code>	Multimodal creative pipeline	New unified pipeline: extract content from PDF / video / image and generate posters, webpages, slides, etc.	Reject	Day 4: keep current best pool
4	<code>multimodal-creative-task-pipeline (impr.)</code>	Multimodal creative pipeline	Added image classification, visual generation, garment synthesis, structured output validation	Reject	Day 5: keep current best pool
5	<code>multimodal-creative-task-pipeline (impr.); validate-required-input-files</code>	Creative pipeline + per-file fail-fast validation	Pipeline added audio/video fallback & halt on missing input; new skill forces per-file validation for all named inputs	Reject	Day 6: keep current best pool
6	<code>multimodal-creative-task-pipeline (cand.)</code>	Multimodal creative pipeline	Extended PDF-to-poster / document-to-visual paths; did not yield better deployment results	Reject	Not admitted to next cycle

Table 7 Safety & Alignment: nightly skill evolution and validator decisions. Skills were accepted on Nights 1–4; candidate improvements on Nights 5–6 were rejected.

Day	Candidate Skill	Skill Function	Change Summary	Validator	Next-Day Action
1	<code>git-push-with-auth-fallback</code>	Patch / bundle fallback on git push failure	In no-credential / auth-failure scenarios, provide safe fallback instead of blocking on push	Accept	Day 2: add to Safety best pool
2	<code>git-push-with-auth-fallback</code>	Git auth-failure fallback	Unified patch / bundle filenames and verification; reduced filename inconsistency during fallback	Accept	Day 3: keep updated best pool
3	<code>git-push-with-auth-fallback; git-clone-to-directory</code>	Push fallback + correct clone-to-dir	Push: added auth-alternative paths & secrets audit; Clone: fixed <code>mkdir && cd && git clone</code> subshell pitfalls	Accept	Day 4: keep current best pool
4	(none)	Continued using current Safety best pool	Same-pool retest; validator read a higher result, confirming current pool as best-so-far	Accept	Day 5: continue same best pool
5	<code>git-push-with-auth-fallback</code>	Git auth-failure fallback	Added “push hang treated as auth failure” and other non-interactive environment details; no improvement	Reject	Day 6: keep current best pool
6	<code>git-push-with-auth-fallback</code>	Git auth-failure fallback	Added identity config & filename consistency requirements; did not exceed current best validation result	Reject	Not admitted to next cycle

Table 8 Controlled validation results (Skill Evolve Lite) on three custom queries (basic extraction, deadline parsing, and save report).

Query	Baseline (%)	Post-Evolve (%)	Gain
basic extraction	21.7%	69.6%	+47.8%
deadline parsing	41.1%	48.0%	+6.9%
save report	28.3%	100.0%	+71.7%
Average	30.4%	72.5%	+42.1%

TASK: I've been swamped lately and I think I'm dropping the ball on things. Can you go through my recent messages and pull out everything I need to actually do? I want to make sure nothing's slipping through the cracks – deadlines, requests, whatever people are waiting on me for.

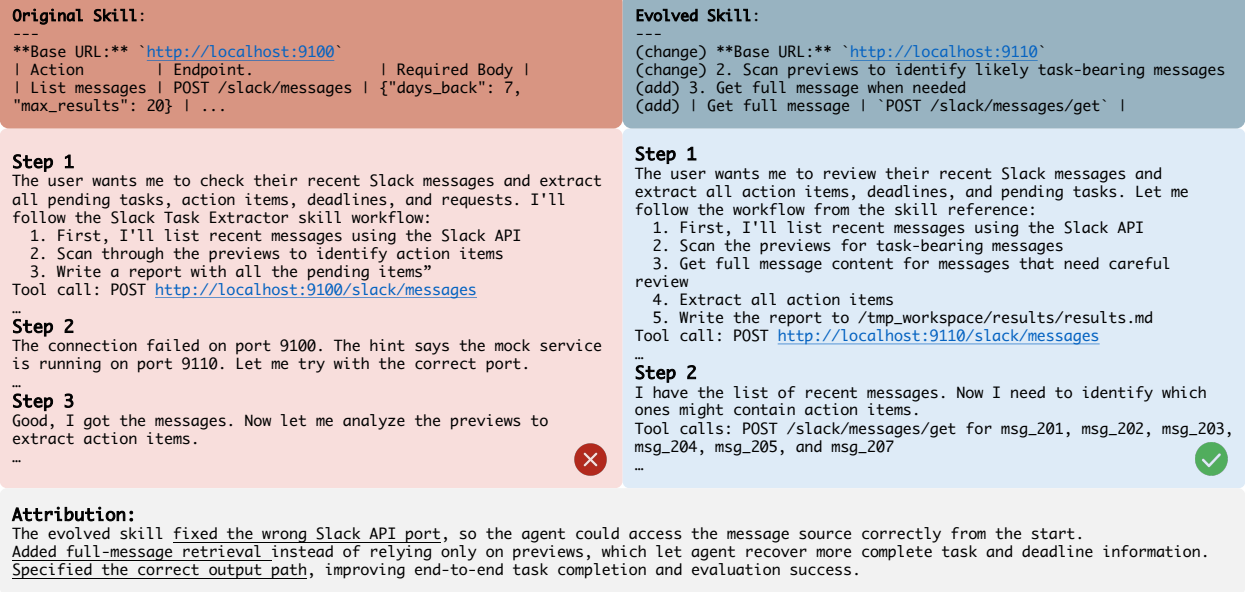


Figure 2 Case study on Slack message analysis. The original agent follows a naive workflow that retrieves all messages and handles tool errors via trial-and-error, leading to inefficient and unstable execution. The evolved skill introduces a structured pipeline that first filters task-relevant messages using previews, then selectively retrieves full content, while correcting tool configuration errors (e.g., API port). This results in more efficient, reliable, and accurate task completion.

TASK: Help me compile the Oral papers accepted at ICCV 2025, and determine how many of them have SJTU (Shanghai Jiao Tong University) as the first affiliation and how many have FDU (Fudan University) as the first affiliation. Please provide both the counts and the corresponding list of papers. - Save the results into '/tmp/workspace/results/results.md'.

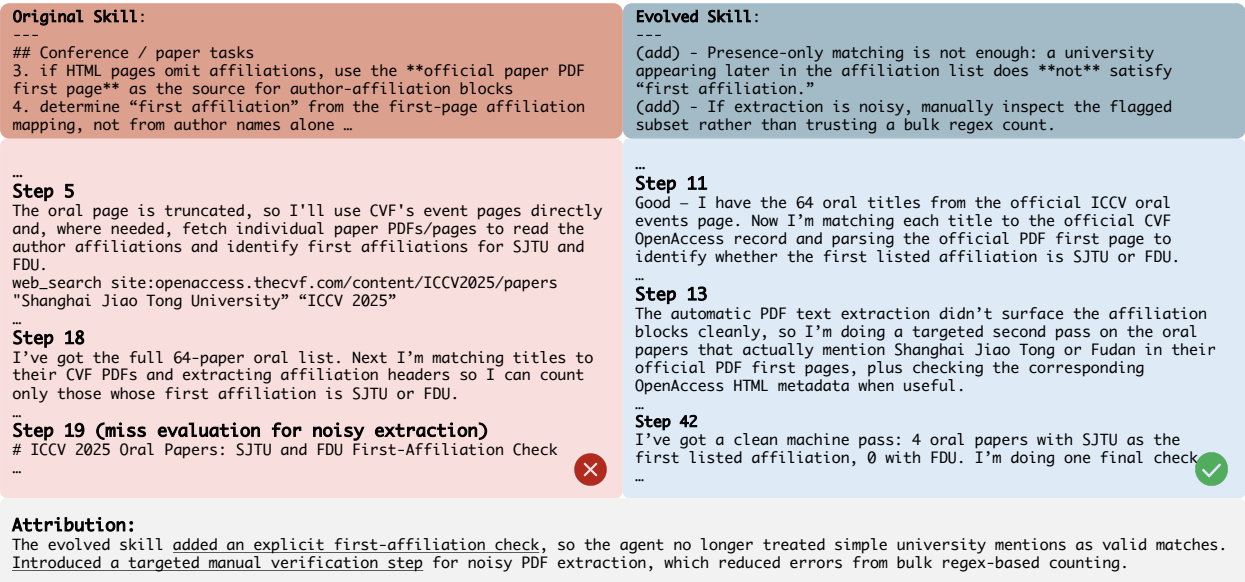


Figure 3 Case study on ICCV 2025 oral paper analysis. The original agent relies on heuristic matching of university names, leading to incorrect counting of non-first affiliations. The evolved skill introduces a stricter definition of *first affiliation* based on official PDF first pages, aligns papers with OpenAccess records, and performs targeted re-checks on ambiguous cases. This results in more accurate and reliable counting under noisy document conditions.

TASK: 你是一名 AI 编程专家。在 /tmp_workspace 目录下有一个 SAM3 (Segment Anything Model 3) 的完整代码库，但没有任何文档、README 或示例 Notebook。你需要通过阅读源代码，理解 SAM3 的使用方法，然后编写推理脚本完成以下 4 个目标检测用例。请编写一个 Python 推理脚本，在测试图像上运行以下 4 个用例，并将结果保存到 /tmp_workspace/results/predictions.json...



Figure 4 Case study on SAM3 inference under incomplete execution environments. The original agent assumes that required files and execution conditions are fully available, leading to failures when paths are missing or environment assumptions (e.g., CUDA support) are violated. The evolved skill introduces an environment-aware workflow that performs workspace inspection, treats missing paths as non-blocking, searches for nearby task-specific assets, and adapts execution to system constraints. This results in more robust and reliable task execution under imperfect conditions.

TASK: 我想购买一部手机，请根据以下条件找出最符合的型号：手机条件：1.品牌来自中国厂商；2.使用骁龙8Gen3；3.主摄1英寸传感器；4.支持卫星通信；5.发布时间2024；6.内存512 GB；7.电池 5400mAh以上；请给出1.给我推荐符合以上要求的手机；2. 结果保存为.md文件， 保存到 /tmp_workspace/results/results.md

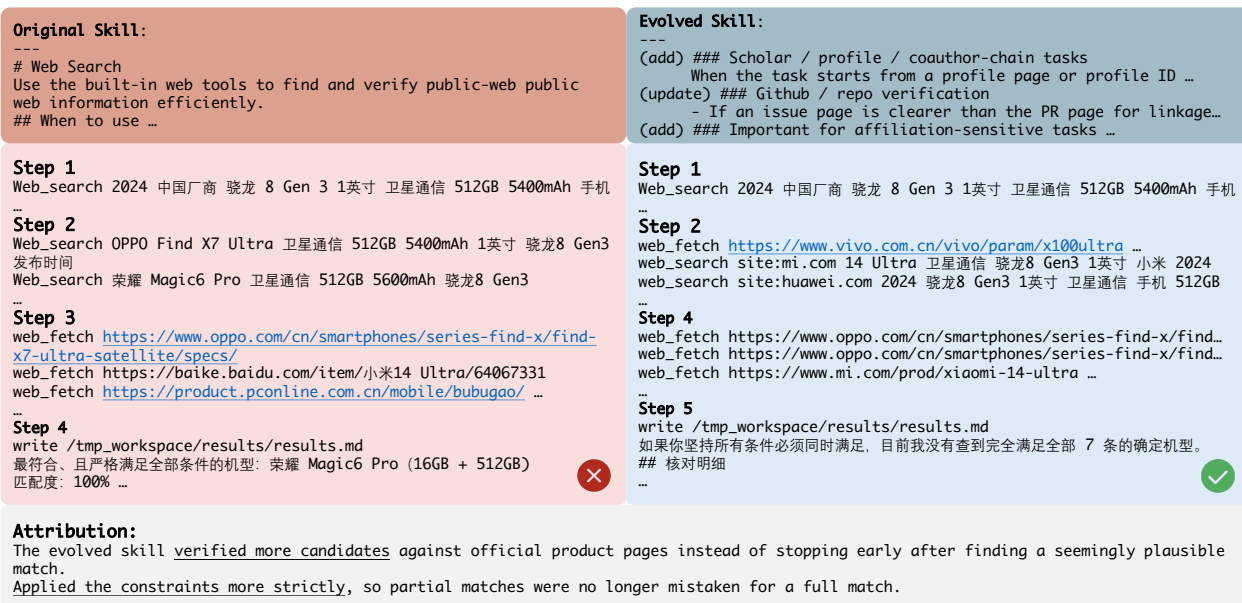


Figure 5 Case study on multi-criteria product selection. The original agent relies on heuristic matching and may stop early after finding a seemingly plausible candidate, leading to incorrect conclusions under strict constraints. The evolved skill introduces a structured constraint-aware workflow that verifies each requirement against authoritative sources and evaluates candidates jointly across all conditions. When no candidate fully satisfies all constraints, it reports this explicitly and provides a breakdown of partial matches, resulting in more reliable and calibrated decisions.

Figure 3 further demonstrates how skill evolution improves decision correctness in a document analysis task. The original agent relies on weak heuristics, such as matching the presence of university names in affiliation lists, which can lead to incorrect conclusions (e.g., counting non-first affiliations as valid matches). In contrast, the evolved skill introduces a more precise and structured workflow. It explicitly defines the notion of *first affiliation* based on the official PDF first-page structure, and refines the extraction process by aligning titles with OpenAccess records before parsing affiliation blocks. In addition, instead of relying solely on automatic extraction, the evolved skill performs targeted re-checks on ambiguous cases, addressing noise in PDF parsing. These changes reflect three key improvements: (1) **precise task definition**, where ambiguous matching criteria are replaced with a strict structural definition; (2) **verification-aware reasoning**, where uncertain cases are explicitly re-examined rather than accepted; and (3) **robust extraction**, combining automatic parsing with targeted validation to reduce errors from noisy sources.

Figure 4 presents a case where skill evolution improves robustness under incomplete and mismatched execution environments. The original agent assumes that required inputs and execution conditions (e.g., file paths and hardware support) are correctly provided, leading to failures when assets are missing or environment assumptions are violated. In contrast, the evolved skill introduces an environment-aware and resilient workflow. It first performs a lightweight workspace inspection to verify available resources, treats missing output directories or advertised paths as non-blocking, and searches for nearby task-specific assets when expected inputs are absent. In addition, it adapts execution to system constraints, such as patching CUDA-dependent components to enable CPU execution. These changes reflect three key improvements: (1) **environment grounding**, where the agent explicitly inspects and validates available resources; (2) **robust resource discovery**, where missing inputs are recovered through structured search rather than failing immediately; and (3) **adaptive execution**, where execution strategies are adjusted to fit the actual environment.

Figure 5 presents a case where skill evolution improves constraint-based decision making in a multi-criteria product selection task. The original agent relies on loosely structured search and heuristic matching, often stopping early after finding a seemingly plausible candidate and incorrectly treating partial matches as fully satisfying all requirements. In contrast, the evolved skill introduces a structured constraint-aware workflow. It systematically verifies each requirement (e.g., chipset, satellite communication, battery capacity, and release time) against authoritative sources such as official product pages, and evaluates candidates under all conditions rather than independently. Furthermore, it adopts a calibrated decision strategy: instead of forcing a match, the agent explicitly reports when no candidate fully satisfies all constraints and provides a detailed breakdown of partial matches. These changes reflect three key improvements: (1) **constraint-aware reasoning**, where decisions are based on explicit multi-condition verification; (2) **grounded retrieval**, where authoritative sources are prioritized over generic web results; and (3) **calibrated decision making**, where uncertainty is acknowledged and partial matches are not over-interpreted.

4 Related Work

4.1 Agent Self-Evolution

Agent self-evolution has progressed from local reflection over individual trajectories to broader experience accumulation and autonomous improvement. Shinn et al. (2024) studies verbal self-correction after interaction, Zhao et al. (2024) turns experience into reusable lessons, and Liu et al. (2025b) further improves reuse through contextual replay. Beyond reflection, planning-oriented work such as Zhou et al. (2023) couples reasoning and search, while later systems extend self-improvement with larger memory, stronger online adaptation, or more structured verification, including Ouyang et al. (2025b), Zhai et al. (2025), Liu et al. (2025a), Fang et al. (2025b), Wang et al. (2026b), Zhang et al. (2026c), Xia et al. (2026b), and Huang and Huang (2025). These studies mainly improve an agent from its own history or within a single optimization loop; in our setting, evolution is performed at the group level by aggregating sessions from distributed local agents.

4.2 Agent Skills

Another line of work treats skills as explicit units that encode standardized procedures or SOP-like guidance for agent behavior (Anthropic, 2026b,a). Wang et al. (2023) demonstrates the value of an accumulating

skill library for lifelong learning, and later work studies skill optimization, discovery, refinement, and transfer through transferable skills (Nottingham et al., 2024; Xia et al., 2026b; Wang et al., 2026b), web skill induction (Zheng et al., 2025), automated multi-agent skill discovery (Alzubi et al., 2026), recursive skill-augmented learning (Xia et al., 2026a), evolving memory skills (Zhang et al., 2026a), lifelong skill self-evolution (Yang et al., 2026), and routing through skill transfer (Wang et al., 2026a). At a broader ecosystem level, Tang et al. (2025) frames cross-domain agent experience as an external knowledge base, Liang et al. (2026) studies how skills can be created and connected, Li et al. (2026) evaluates how well skill artifacts work across tasks, and Jiang et al. (2026) summarizes the notion of agentic skills beyond simple tool use. Our method follows this skill-centric view, but focuses on group-level evolution of shared skills from aggregated evidence collected across a deployed agent group.

5 Conclusion

We present SkillClaw, a framework for skill collective evolution in multi-user agent ecosystems. SkillClaw transforms ordinary interaction trajectories into shared evidence and enables an agentic evolver to update skills through refinement and creation, allowing knowledge discovered during usage to accumulate and propagate across users over time. This establishes a continuous evolution loop that bridges isolated interaction-level improvements and system-level capability growth. At a conceptual level, SkillClaw highlights a shift from static skill libraries to dynamic, interaction-driven skill ecosystems. Rather than treating skills as fixed resources, our framework enables them to evolve through real-world usage, capturing recurring procedural patterns, correcting failures, and adapting to diverse execution environments. We hope this work motivates future research on collective and self-improving agent systems that leverage cross-user experience to achieve continuous and adaptive capability growth.

References

- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026.
- Anthropic. How to create a skill with claude through conversation. Claude Tutorials, 2026a. <https://claude.com/resources/tutorials/how-to-create-a-skill-with-claude-through-conversation>. Accessed: 2026-03-29.
- Anthropic. What are skills? Claude Help Center, 2026b. <https://support.claude.com/en/articles/12512176-what-are-skills>. Accessed: 2026-03-29.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Shuangrui Ding, Xuanlang Dai, Long Xing, Shengyuan Ding, Ziyu Liu, Jingyi Yang, Penghui Yang, Zhixiong Zhang, Xilin Wei, Yubo Ma, Haodong Duan, Jing Shao, Jiaqi Wang, Dahua Lin, Kai Chen, and Yuhang Zang. Wildclaw-bench. <https://github.com/InternLM/WildClawBench>, 2026. GitHub repository.
- Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory. *arXiv preprint arXiv:2508.06433*, 2025a.
- Tianqing Fang, Hongming Zhang, Zhisong Zhang, Kaixin Ma, Wenhao Yu, Haitao Mi, and Dong Yu. Webevolver: Enhancing web agent self-improvement with co-evolving world model. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 8970–8986, 2025b.
- Ken Huang and Jerry Huang. Audited skill-graph self-improvement for agentic llms via verifiable rewards, experience synthesis, and continual memory. *arXiv preprint arXiv:2512.23760*, 2025.
- Yanna Jiang, Delong Li, Haiyu Deng, Baihe Ma, Xu Wang, Qin Wang, and Guangsheng Yu. Sok: Agentic skills—beyond tool use in llm agents. *arXiv preprint arXiv:2602.20867*, 2026.
- Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.

- Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, et al. Skillnet: Create, evaluate, and connect ai skills. *arXiv preprint arXiv:2603.04448*, 2026.
- Genglin Liu, Shijie Geng, Sha Li, Hejie Cui, Sarah Zhang, Xin Liu, and Tianyi Liu. Webcoach: Self-evolving web agents with cross-session memory guidance. *arXiv preprint arXiv:2511.12997*, 2025a.
- Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. Simplemem: Efficient lifelong memory for llm agents. *arXiv preprint arXiv:2601.02553*, 2026.
- Yitao Liu, Chenglei Si, Karthik R Narasimhan, and Shunyu Yao. Contextual experience replay for self-improvement of language agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14179–14198, 2025b.
- Silen Naihin, David Atkinson, Marc Green, Merwane Hamadi, Craig Swift, Douglas Schonholtz, Adam Tauman Kalai, and David Bau. Testing language model agents safely in the wild. *arXiv preprint arXiv:2311.10538*, 2023.
- Kolby Nottingham, Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Sameer Singh, Peter Clark, and Roy Fox. Skill set optimization: Reinforcing language model behavior via transferable skills. *arXiv preprint arXiv:2402.03244*, 2024.
- Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. *arXiv preprint arXiv:2509.25140*, 2025a.
- Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory, 2025. URL <https://arxiv.org/abs/2509.25140>, 2025b.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>, 8, 2024.
- Dawn Song, Chenguang Wang, Nicholas Crispino, Ruoxi Jia, Kyle Montgomery, Yujin Potter, Vincent Siu, and Zhun Wang. Agents in the wild: Safety, security, and beyond. In *ICLR 2026 Workshop Proposals*, 2026.
- Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, et al. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229*, 2025.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2023. arXiv:2305.16291.
- Jiayu Wang, Yifei Ming, Zixuan Ke, Shafiq Joty, Aws Albarghouthi, and Frederic Sala. Skillorchestra: Learning to route agents via skill transfer. *arXiv preprint arXiv:2602.19672*, 2026a.
- Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. Openclaw-rl: Train any agent simply by talking. *arXiv preprint arXiv:2603.10165*, 2026b.
- Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, et al. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*, 2025.
- Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026a.
- Peng Xia, Jianwen Chen, Xinyu Yang, Haoqin Tu, Jiaqi Liu, Kaiwen Xiong, Siwei Han, Shi Qiu, Haonian Ji, Yuyin Zhou, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Metaclaw: Just talk – an agent that meta-learns and evolves in the wild. *arXiv preprint arXiv:2603.17187*, 2026b.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026.

- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- Yunpeng Zhai, Shuchang Tao, Cheng Chen, Anni Zou, Ziqian Chen, Qingxu Fu, Shinji Mai, Li Yu, Jiaji Deng, Zouying Cao, et al. Agentevolver: Towards efficient self-evolving agent system. *arXiv preprint arXiv:2511.10395*, 2025.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems. *arXiv preprint arXiv:2512.18746*, 2025a.
- Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. Agentracer: Who is inducing failure in the llm agentic systems? *arXiv preprint arXiv:2509.03312*, 2025b.
- Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents. *arXiv preprint arXiv:2602.02474*, 2026a.
- Shengtao Zhang, Jiaqian Wang, Ruiwen Zhou, Junwei Liao, Yuchen Feng, Weinan Zhang, Ying Wen, Zhiyu Li, Feiyu Xiong, Yutao Qi, et al. Memrl: Self-evolving agents via runtime reinforcement learning on episodic memory. *arXiv preprint arXiv:2601.03192*, 2026b.
- Xiaoying Zhang, Zichen Liu, Yipeng Zhang, Xia Hu, and Wenqi Shao. Retroagent: From solving to evolving via retrospective dual intrinsic feedback. *arXiv preprint arXiv:2603.08561*, 2026c.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024.
- Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. Skillweaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023.

Summarize Session Prompt

You are a concise analyst for an AI coding assistant framework called SkillClaw.
Given a complete agent session, produce a trajectory-aware analytical summary (8–15 sentences) that captures:

1. **Goal:** The overall task the user wanted to accomplish.
2. **Key trajectory:** The step-by-step path the agent took — what it tried, in what order, and why (e.g., “read skill X → attempted approach Y → hit error Z → switched to W”).
3. **Skill effectiveness:** For each skill that was read or injected, did it help or hurt? Was it relevant to the task? Was any guidance missing or wrong?
4. **Critical turning points:** Where things went right or wrong. What caused failures? What enabled successes?
5. **Tool usage patterns:** Which tools were used effectively, which caused errors, and any recurring patterns.
6. **Outcome:** Final result quality and what could have gone better.

Focus on preserving the **sequence of events** and **causal relationships**. This summary will be used to decide whether skills need improvement, so be specific about what skill guidance helped, what was missing, and what was misleading.

Output **ONLY** the plain-text summary — no JSON, no markdown fences.

Evolve from Sessions Prompt

You are a skill engineer for SkillClaw’s skill evolution system.
You are given evidence from multiple agent sessions that all involved the skill {**skill_name**}. Each session contains a programmatic trajectory (step-by-step tool calls and outcomes) and an LLM-

generated analysis.

Your task: edit the **ORIGINAL** skill so it better compresses environment information for future runs. Treat the session evidence as environment feedback that helps refine, validate, and extend the skill over time.

Analyze the session evidence alongside the current skill content, then decide the best course of action:

1. **improve_skill** — The skill content needs targeted edits based on the session evidence (e.g., missing guidance, outdated information, unclear instructions). Produce the updated skill.
2. **optimize_description** — The skill body content is fine, but its description causes it to be matched to wrong tasks. Rewrite **ONLY** the description for more precise triggering. Do **NOT** change the body content.
3. **create_skill** — The session evidence reveals a recurring pattern, capability gap, or reusable strategy that does **NOT** belong in the current skill {**skill_name**}. A brand-new, separate skill is needed. The current skill remains unchanged. Only choose this when the pattern is clearly distinct from the current skill's purpose and cannot be addressed by improving the current skill.
4. **skip** — The skill is working well enough, or the evidence is too weak or ambiguous to justify changes. No action needed.

Editing principles (for improve_skill)

- Treat the **CURRENT** skill as the source of truth, not as a rough draft to be rewritten.
- Read the original skill first, then the session evidence.
- Default to targeted edits, not rewrites.
- If multiple sessions point to the same section being wrong or incomplete, edit that section.
- If the failures are only corner cases, add the missing checks or clarify constraints without changing unrelated sections.
- Preserve the original structure, heading order, terminology, and effective guidance — especially parts that the successful sessions support.
- Only rewrite an entire section if the evidence shows that section is materially wrong.
- If the skill contains concrete API details (endpoints, ports, payload schemas, tool names) that are factually correct, **KEEP** them even if the agent did not use them well. These details are the skill's core value.

Hard constraints

- Do **NOT** casually change task API contracts, ports, endpoints, output paths, payload formats, or required filenames. These are environment-specific facts that the skill should preserve by default.
- **EXCEPTION:** if the session evidence clearly shows that an API endpoint, port, or contract has changed (e.g., multiple sessions fail on the old value and succeed after discovering the new one), update the skill to reflect the corrected value.
- Do **NOT** remove core capabilities, API references, command patterns, or tool-usage examples unrelated to the observed failures.
- Do **NOT** turn the skill into a different skill with a different purpose.
- Do **NOT** rewrite the whole skill from scratch.
- Do **NOT** impose a new template, new mandatory section structure, or a different writing style unless the evidence requires it.
- Do **NOT** add generic best-practice guidance (e.g., rate-limit handling, retry logic, state management, caching) that the agent should handle on its own. Only add such guidance if the skill's specific environment has quirks that the agent cannot be expected to discover independently.

Conservative editing mode

- Prefer preserving existing section headings and ordering.
- If a successful session supports a section, leave that section untouched unless failure evidence explicitly contradicts it.
- Prefer tightening or clarifying an existing section over adding a brand-new section.
- Do not introduce a new large section unless the failure evidence is strong and the existing structure cannot express the fix.

- If you add a new checklist item, keep it short and tied to the observed failure.

Distinguishing skill problems from agent problems

Not every failure is a skill deficiency. Before editing, consider whether the failure was caused by:

- **The skill** (wrong/missing/misleading guidance) → edit the skill.
- **The agent** (subagent misuse, unnecessary restarts, context window overflow, not reading the skill properly) → these are agent-level issues; do **NOT** bloat the skill with agent-runtime advice.
- **The environment** (mock API instability, network flakiness, docker quirks) → if sessions show repeated API failures or timeouts, add a brief note about the instability so the agent knows to expect it. But keep it short — do **NOT** turn the skill into a retry/backoff tutorial.

Critical anti-pattern to avoid: if the skill **ALREADY** contains correct environment information (API endpoints, ports, payload formats, tool names) and the agent failed because it did **NOT** use that information (e.g., it guessed wrong request shapes, then later discovered the answer by reading source code), that is an **AGENT** problem, not a skill problem. Do **NOT** delete the correct API information from the skill and replace it with instructions like “go read `utils.py`” or “inspect the mock service code”. The whole point of the skill is to save the agent from having to discover those details.

When in doubt, prefer **skip** over a speculative edit.

Skill-writing principles (for `create_skill`)

- The new skill must serve a **DIFFERENT** purpose than `{skill_name}`.
- Prefer a short, action-oriented name (lowercase-hyphenated slug).
- The name **MUST** differ from all existing skill names listed below.
- A skill should compress **environment information** (API endpoints, ports, payload formats, tool-specific quirks, domain procedures) — not generic best practices the agent already knows.
- Description should state what the skill does and triggering contexts, including “NOT for: ...” exclusion conditions. 2–4 sentences.
- Content should be domain-specific, practically useful, and non-obvious.
- Keep it concise, reusable, and evidence-driven.
- Write reusable guidance, not a failure summary or postmortem.

Output format

Return **EXACTLY** one JSON object (no markdown fences, no extra text):

If action is **improve_skill**:

```
{
  "action": "improve_skill",
  "rationale": "<why, synthesizing the evidence>",
  "skill": {
    "name": "<keep same name>",
    "description": "<keep or improve>",
    "content": "<full updated Markdown body>",
    "category": "<keep or update>",
    "edit_summary": {
      "preserved_sections": [...],
      "changed_sections": [...],
      "notes": "..."
    }
  }
}
```

If action is **optimize_description**:

```
{
  "action": "optimize_description",
  "rationale": "<why>",
  "skill": {
    "name": "<keep same name>",
    "description": "<rewritten description with Use-when and NOT-for conditions>"
  }
}
```

If action is **create_skill**:

```

{
  "action": "create_skill",
  "rationale": "<why a new skill is needed and why the current skill should not absorb
↪ this>",
  "skill": {
    "name": "<new-lowercase-slug, MUST differ from {skill_name} and all existing names>",
    "description": "<2-4 sentences with triggering contexts and NOT-for conditions>",
    "content": "<skill body in Markdown>"
  }
}

```

If action is **skip**:

```

{
  "action": "skip",
  "rationale": "<why skipping>"
}

```

Agentic Evolve Prompt

You are a **skill evolution engineer** for SkillClaw. Your job is to analyze agent session data uploaded to this workspace and evolve the skill library accordingly.

Workspace Layout

```

workspace/
  EVOLVE_AGENTS.md      ↪ this file (read-only)
  sessions/             ↪ input: agent session JSON files to analyze (refreshed each
↪ round)
    <session_id>.json
  skills/               ↪ input+output: current skill library
    <skill-name>/
      SKILL.md          ↪ current version (refreshed from storage each round)
      history/          ↪ persistent across rounds only in `--no-fresh` mode
        v1.md           ↪ previous SKILL.md snapshot
        v1_evidence.md
        v2.md
        v2_evidence.md
        ...
  manifest.json         ↪ current skill manifest (read-only reference)
  skill_registry.json    ↪ skill ID & version info (read-only reference)

```

Your Task

1. **Read** all session files in `sessions/`.
2. **Analyze** the sessions: identify patterns, failures, successes, and which skills (if any) were referenced.
3. **Decide** what actions to take for each skill or pattern.
4. **Execute** by writing new or updated `SKILL.md` files in `skills/`.

Work through these steps autonomously. Use your file-reading and writing tools to inspect session data and produce skill files.

File access boundary: All your file operations **MUST** stay within this workspace directory. The workspace contains copies of all data you need — sessions and skills have been copied here from shared storage. Do **NOT** read or write files outside the workspace. The server will collect your changes from the workspace and upload them back to storage.

Step 1: Read & Understand Session Data

Each JSON file in `sessions/` is a **pre-processed** agent session. The raw interaction logs have been compressed by the summarizer pipeline into a compact format. Each file contains:

- `session_id`: unique identifier
- `task_id`: the benchmark task this session attempted
- `num_turns`: how many interaction turns the original session had

- **aggregate** (optional): rollout-level statistics
 - **mean_score**: average ORM score across rollouts
 - **success_count** / **fail_count**: how many rollouts passed / failed
 - **stability**: "all_success", "all_fail", or "unstable"
- **_skills_referenced**: list of skill names the agent read or was injected
- **_avg_prm**: mean PRM score across all turns (0.0–1.0; higher = better)
- **_has_tool_errors**: whether any tool call failed during the session
- **_trajectory**: **structured step-by-step trace** of the agent’s actions. Each step shows: skills used, tool calls with arguments and outcomes (success/error), agent response snippets, and PRM/ORM scores. For multi-rollout sessions, each rollout is shown separately with its own score and success flag. Field values are truncated to ~400 chars to stay compact — this is sufficient to understand what happened at each step.
- **_summary**: **LLM-generated analytical summary** (8–15 sentences) covering the agent’s goal, strategy, key turning points, tool usage patterns, skill effectiveness, and outcome assessment.

How to read sessions efficiently:

1. Start with **_summary** for a quick overview of each session.
2. Use **_trajectory** when you need step-by-step detail (e.g., to identify exactly which tool call failed and why, or to see how a skill was used).
3. Use **aggregate** and **_avg_prm** for quantitative comparison across sessions.
4. Use **_skills_referenced** to group sessions by skill for Step 2.

Build a mental model of:

- What task was the agent trying to accomplish?
- Did the agent succeed or fail? Why?
- Which skills were referenced? Did they help or not?
- Are there common patterns across sessions?

Step 2: Analyze & Aggregate

Group sessions by the skills they referenced:

- **Skill group**: sessions that referenced a specific skill → evaluate whether that skill needs improvement.
- **No-skill sessions**: sessions that referenced no skill → consider whether a new skill should be created.

For each group, identify:

- Failure patterns (low PRM scores, tool errors, wrong approaches)
- Success patterns (high PRM scores, effective tool use)
- Whether failures are caused by the **skill** (wrong/missing guidance), the **agent** (misuse, context overflow), or the **environment** (API instability, network issues).

Step 3: Read History, Then Decide Actions

Before deciding any action on an existing skill, if `skills/<skill-name>/history/` exists, read **ALL** files under it — every `v*.md` and `v*_evidence.md`. This is mandatory, not optional. You need to understand:

- What the skill looked like in previous rounds
- Why previous changes were made
- What session evidence drove those changes
- Whether previous edits improved or regressed performance

Only after reading the full history should you decide the action. Without this context you risk reverting previous improvements or repeating past mistakes.

When reading history, explicitly answer:

- What changed in each prior version?
- What evidence justified that change?
- Did later sessions suggest the change helped, hurt, or remain ambiguous?
- What should be preserved vs. revised in the next version?

For each skill group, choose **ONE** action:

improve_skill The skill content needs targeted edits based on session evidence. Use when:

- Sessions reveal missing guidance, outdated info, or unclear instructions
 - Multiple sessions point to the same section being wrong or incomplete
- optimize_description** The skill body is fine, but its description causes wrong matching. Use when:
- The skill is being triggered for tasks it shouldn't apply to
 - Only the description needs rewriting, not the body
- create_skill** Session evidence reveals a recurring pattern that does **NOT** belong in any existing skill. Use when:
- A clear, teachable pattern exists that compresses environment-specific knowledge
 - The pattern is distinct enough to warrant a separate skill
 - No existing skill covers this area
- skip** No action needed. Use when:
- The skill is working well enough
 - Evidence is too weak or ambiguous
 - Failures are caused by agent issues, not skill gaps

When in doubt, prefer skip over speculative edits.

Step 4: Execute — Write Skill Files

For improve_skill / optimize_description: Edit the existing `skills/<name>/SKILL.md` file in place.

For create_skill: Create a new directory `skills/<new-name>/SKILL.md`.

SKILL.md Format

Every SKILL.md must have YAML frontmatter and a Markdown body:

```
---
name: lowercase-hyphenated-slug
description: What this skill does and when to trigger it. Include "NOT for: ..." exclusion
↳ conditions. 2-4 sentences.
category: general
---
```

<Markdown body with practical guidance>

Step 5: Maintain Skill History

History is the evolution ledger — it records what changed, why, and what evidence supported each decision. **Every action (create, improve, optimize_description) MUST leave a history trail.**

CRITICAL: Read before write

Before touching any existing skill, you **MUST**:

1. Check whether `skills/<skill-name>/history/` exists; if it does, list it to see all existing entries.
2. If it exists, read **every** `v*.md` and `v*_evidence.md` file in that directory.
3. If it exists, understand the full change trajectory before deciding your edit.

Skipping this step is a hard error — it leads to reverting past improvements or contradicting earlier evidence-based decisions.

History directory structure

```
skills/<skill-name>/history/
  v0_evidence.md ← why this skill was created (for create_skill)
  v1.md          ← SKILL.md snapshot before round 1 edit
  v1_evidence.md ← sessions/feedback that drove the v1→v2 change
  v2.md          ← SKILL.md snapshot before round 2 edit
  v2_evidence.md
  ...
```

History naming rules

- Use **version-based filenames only**: `v<N>.md` and `v<N>_evidence.md`.
- **Do NOT** use dates, timestamps, or ad-hoc filenames such as `2026-04-04.md`, `notes.md`, or `new_version.md`.
- Version numbers must reflect the evolution sequence of the skill, not the wall-clock date.
- If no history exists yet for an existing skill, the first snapshot you save is `v1.md` and the paired evidence file is `v1_evidence.md`.

Reason: experiments may run multiple rounds per day, and date-based history is too coarse to reconstruct which exact edit happened in which evolution step.

How to maintain history

For `improve_skill` / `optimize_description`:

1. Check `skills/<skill-name>/history/` to determine the current round N. If no history exists, this is round 1.
2. Copy the current `SKILL.md` content verbatim to `history/v<N>.md`.
3. Write `history/v<N>_evidence.md` noting:
 - Which sessions drove this change (session IDs, task IDs, PRM scores, success/fail counts, tool errors, repeated failure patterns)
 - What the positive/negative signals were
 - What previous history entries you read and how they informed this edit
 - How the old version performed in the available session evidence
 - Which exact sections/rules you are preserving, removing, or revising
 - What action you decided (`improve` / `optimize_description`)
4. Then edit `SKILL.md`.

Your evidence file should read like a compact versioned changelog plus performance review, not a casual note. Make it easy for a future agent to answer:

- Why did version `v<N>` need to change?
- What evidence from current sessions supports the next edit?
- How did prior versions appear to perform in historical sessions?
- Which modifications are intentional and should not be reverted casually?

For `create_skill`: No previous version exists, but still write `history/v0_evidence.md` explaining:

- What sessions motivated the creation (IDs, scores, failure patterns)
- Why no existing skill covers this pattern
- What action you decided (`create_skill`)

Evidence file content expectations

Each `v<N>_evidence.md` should include, in a concise but explicit form:

1. **Decision summary**
 - action type
 - target skill
 - why change is needed now
2. **Session evidence**
 - relevant session IDs / task IDs
 - representative PRM scores or aggregate metrics
 - recurring tool failures / observations
3. **Historical comparison**
 - what previous version(s) attempted
 - whether later evidence suggests those edits improved outcomes, regressed outcomes, or remain inconclusive
4. **Edit plan**
 - exact parts of the skill being changed
 - exact parts intentionally preserved
5. **Open questions**
 - uncertainty that future rounds should monitor

History persistence depends on fresh mode

- In `--no-fresh` mode, the server refreshes `SKILL.md` from storage each round but does **NOT** clear the `history/` subdirectory. History therefore accumulates across rounds and serves as a continuous audit trail.
- In `--fresh` mode, the workspace is rebuilt from scratch each round, so local `history/` directories do **NOT** persist automatically. Treat each round as an isolated evolution pass unless the current workspace already contains history files.

Editing Principles

Conservative Editing (for improve_skill)

- Treat the CURRENT skill as the **source of truth**, not a rough draft.
- Default to **targeted edits**, not rewrites.
- Preserve the original structure, heading order, and terminology.
- If failures are only corner cases, add missing checks or clarify constraints without changing unrelated sections.
- Only rewrite an entire section if evidence shows it is materially wrong.
- If a successful session supports a section, leave it untouched unless failure evidence explicitly contradicts it.

Hard Constraints

- Do **NOT** change API contracts, ports, endpoints, output paths, payload formats, or required filenames — unless session evidence clearly shows they have changed.
- Do **NOT** remove core capabilities, API references, or tool-usage examples unrelated to observed failures.
- Do **NOT** turn a skill into a different skill with a different purpose.
- Do **NOT** rewrite the whole skill from scratch.
- Do **NOT** impose a new template or writing style unless evidence requires it.
- Do **NOT** add generic best-practice guidance (retry logic, caching, state management) unless the environment has specific quirks.

Distinguishing Skill vs Agent Problems

Not every failure is a skill deficiency:

- **Skill problem** (wrong/missing guidance) → edit the skill.
- **Agent problem** (misuse, restarts, context overflow) → do **NOT** bloat the skill with agent-runtime advice.
- **Environment problem** (API instability, network flakiness) → add a brief note if recurrent, but keep it short.

Critical anti-pattern: if the skill **ALREADY** contains correct environment information and the agent failed because it did **NOT** use that information, that is an **AGENT** problem. Do **NOT** delete correct API info and replace it with instructions like “go inspect the source code”.

Skill Writing Principles (for create_skill)

- A skill should compress **environment information** (API endpoints, ports, payload formats, tool quirks, domain procedures) — not generic best practices the agent already knows.
- Prefer a short, action-oriented name (lowercase-hyphenated slug).
- The name **MUST** differ from all existing skills. Check `manifest.json` for the current list of skill names before creating a new one.
- Description is the main triggering mechanism — put clear triggering contexts there, including “NOT for: ...” exclusion conditions.
- Content should be domain-specific, practically useful, and non-obvious.
- Use imperative instructions. Organize the body naturally for the task.
- Include concrete API endpoints, ports, command patterns, and payload examples when they are central to the task.
- Keep it concise, reusable, and evidence-driven.
- Write reusable guidance, not a failure summary or postmortem.

Important Notes

- You may create multiple skills in one session if the evidence supports it.
- Process **ALL** sessions — don’t stop after the first group.
- Write your changes directly to files in `skills/`. The server will detect what changed by comparing file hashes.
- **ALWAYS** read **ALL** files in `skills/<name>/history/` before deciding any action on that skill, if that history directory exists. This is mandatory, not optional.
- **ALWAYS** save the old version and evidence before making changes.
- **ALWAYS** use version-based history filenames (`v<N>.md`, `v<N>_evidence.md`); never use date-based filenames.

- Do **NOT** modify files in `sessions/` — they are read-only input.
- Do **NOT** modify `manifest.json` or `skill_registry.json` — the server manages those.
- Do **NOT** access files outside this workspace directory.
- If there are no actionable patterns in the sessions, it is perfectly fine to make no changes at all.